Confirmation Workflow - Striae Implementation Flow

Striae Authenticated Confirmation System - Code-Based Flow Diagrams

Originally released with Striae v0.9.22-beta (09/22/2025), this page has been updated for the v3.0.0 security remediation documented in GHSA-mmf8-487q-p45m.

As of v3.0.0, Striae no longer relies on hash-only trust for confirmation workflows. Export and import paths now require server-issued signatures on forensic manifests and confirmation payloads, with fail-closed validation when signature metadata is missing or invalid.

This page focuses on workflow execution and validation checkpoints. For canonical signing payload rules, signature envelopes, and signing endpoint contracts, see Manifest and Confirmation Signing.

Video Walkthrough

Diagram 1: Mark for Confirmation & Case Export (Original Examiner)

Phase 1-2: Original examiner marks images and exports case package.

```
flowchart TD
    Start([Confirmation Workflow Begins])

    Start --> Phase1["PHASE 1: Mark for Confirmation<br/>(Original
Examiner)"]

    Phase1 --> OE1["OE opens image in Canvas<br/>views Image Notes
sidebar"]
    OE1 --> OE2["OE toggles Include confirmation<br/>checkbox in Notes
sidebar"]
    OE2 --> AnnotData["System sets in AnnotationData:
<br/>includeConfirmation: true"]
    AnnotData --> OE3["OE completes analysis:<br/>- Box annotations<br/>-
Classifications<br/>- Detailed notes"]
    OE3 --> UpdateData["updateCaseData() stores<br/>AnnotationData
with<br/>includeConfirmation flag"]
    UpdateData --> Phase2["PHASE 2: Case Export<br/>(Original Examiner)"]

    Phase2 --> OE4["OE navigates to Case Export<br/>sidebar component"]
    OE4 --> OE5["Selects export format:<br/>ZIP Package + JSON data"]
    OE5 --> OE6["Ensures Include Images<br/>is selected"]
    OE6 --> OE7["Clicks Export Case button"]
    OE7 --> ExportLogic["caseExport component triggers:<br/>onExport
handler"]
    ExportLogic --> BuildPackage["exportCase() action builds:<br/>- case-
data.json<br/>- All evidence images<br/>- Export metadata"]
    BuildPackage --> BuildManifest["generateForensicManifestSecure()
<br/>creates forensic manifest:<br/>- dataHash<br/>- imageHashes<br/>-
manifestHash"]
    BuildManifest --> SignManifest["signForensicManifest()
requests<br/>server-issued signature from DATA_WORKER<br/>returns:<br/>-
manifestVersion: 2.0<br/>- signature {algorithm, keyId, signedAt, value}"]
    SignManifest --> Package["ZIP file created with:<br/>-
FORENSIC_MANIFEST.json (signed)<br/>- case data + images<br/>- read-only
instructions"]
    Package --> AuditExport["auditService.logCaseExport():<br/>- action:
export<br/>- workflowPhase: case-export<br/>- exportedByUid: OE UID<br/>-
hashValid: true"]
    AuditExport --> TransferStart["Transfer ZIP to RE<br/>via secure
channels"]
    TransferStart --> Complete1([Phase 1-2 Complete])

    style Complete1 fill:#90EE90,color:#000
```

Diagram 2: Case Import with Security Validations (Reviewing Examiner)

Phase 3: Reviewing examiner imports case package with comprehensive validations.

```
flowchart TD
    Start([RE Case Import Begins])

    Start --> Phase3["PHASE 3: Case Import<br/>(Reviewing Examiner)"]

    Phase3 --> RE1["RE opens Striae with own<br/>Firebase Auth
credentials"]
    RE1 --> RE2["RE navigates to Case Import<br/>sidebar component"]
    RE2 --> RE3["RE selects received ZIP<br/>package file"]
    RE3 --> Preview["loadCasePreview() reads ZIP:<br/>- Validates ZIP
structure<br/>- Shows case details to RE"]
    Preview --> ConfirmDialog["ConfirmationDialog component<br/>displays
case metadata:<br/>- Case number<br/>- Original examiner info<br/>- Export
date<br/>- Integrity + signature status"]
    ConfirmDialog --> RE4["RE reviews preview<br/>clicks Confirm Import"]
    RE4 --> ManifestCheck{"FORENSIC_MANIFEST.json<br/>present?"}

    ManifestCheck -->|No| ManifestError["[BLOCKED] Missing forensic
manifest<br/>Import cannot proceed"]
    ManifestError --> ErrorEnd1([Import Rejected])
```

```
    ManifestCheck -->|Yes| ValidateSig["verifyForensicManifestSignature():
<br/>- Signature required<br/>- manifestVersion must be 2.0<br/>- Public
key verification by keyId"]
    ValidateSig --> SigCheck{"Signature<br/>valid?"}

    SigCheck -->|No| SigError["[BLOCKED] Signature validation
failed<br/>Missing/invalid signature metadata"]
    SigError --> ErrorEnd2([Import Rejected])

    SigCheck -->|Yes| ValidateIntegrity["validateCaseIntegritySecure():
<br/>- Validate data hash<br/>- Validate image hashes<br/>- Validate
manifest hash"]
    ValidateIntegrity --> IntegrityCheck{"Comprehensive<br/>integrity
valid?"}

    IntegrityCheck -->|No| IntegrityError["[BLOCKED] Integrity validation
failed<br/>File corrupted or tampered"]
    IntegrityError --> ErrorEnd3([Import Rejected])

    IntegrityCheck -->|Yes| Validate3["validateExporterUid():<br/>- Fetch
exporter from USER_WORKER<br/>- Check exists in database"]
    Validate3 --> ExistsCheck{"Exporter UID<br/>exists?"}

    ExistsCheck -->|No| ExistsError["[BLOCKED] Exporter not found<br/>in
database"]
    ExistsError --> ErrorEnd4([Import Rejected])

    ExistsCheck -->|Yes| Validate4["Check if exporter is<br/>same as
current user:<br/>exporterUid === user.uid"]
    Validate4 --> SelfCheck{"Same examiner<br/>as original?"}

    SelfCheck -->|Yes| SelfError["[BLOCKED] SECURITY<br/>Cannot confirm
own work"]
    SelfError --> ErrorEnd5([Import Rejected])

    SelfCheck -->|No| ReadOnlyMode["[SUCCESS] Validations
passed<br/>Import READ-ONLY"]
    ReadOnlyMode --> SetReadOnly["Set case properties:<br/>isReadOnly:
true<br/>originalExaminerId: OE UID"]
    SetReadOnly --> StoreReadOnly["Store in user R2:<br/>READ-ONLY case
directory<br/>originalImageIds mapping"]
    StoreReadOnly --> AuditImport["auditService.logCaseImport():<br/>-
action: import<br/>- workflowPhase: case-import<br/>- hashValid:
true<br/>- signature: present/valid/keyId<br/>- originalExaminerUid: OE
UID"]
    AuditImport --> Complete2([Phase 3 Complete])

    style ManifestError fill:#ff4444,color:#fff
    style SigError fill:#ff4444,color:#fff
    style IntegrityError fill:#ff4444,color:#fff
    style ExistsError fill:#ff4444,color:#fff
    style SelfError fill:#ff4444,color:#fff
    style ReadOnlyMode fill:#90EE90,color:#000
    style Complete2 fill:#90EE90,color:#000
```

Diagram 3: Independent Review & Confirmation (Reviewing Examiner)

Phase 4-5: Reviewing examiner conducts independent analysis and exports confirmations.

Jump to Confirmation Modal Workflow.

```
flowchart TD
    Start([RE Independent Review Begins])

    Start --> Phase4["PHASE 4: Confirmation<br/>(Reviewing Examiner)"]

    Phase4 --> RE5["RE accesses physical evidence<br/>conducts independent
analysis"]
    RE5 --> RE6["RE reviews OE annotations<br/>in read-only UI<br/>all
controls disabled"]
```

```
controls disabled "]
    RE6 --> Decision{"Conclusions<br/>confirmed?"}

    Decision -->|No| NoConfirm["RE does not confirm<br/>Case reviewed but
no<br/>confirmation recorded"]
    NoConfirm --> End1([Process Ends - No Confirmation])

    Decision -->|Yes| RE7["RE clicks Confirm button<br/>in Canvas
component"]
    RE7 --> Modal["ConfirmationModal opens:<br/>ConfirmationModalProps"]
    Modal --> RE8["Modal displays read-only fields:<br/>- RE name from
Auth<br/>- RE email from Auth<br/>- RE company from UserData"]
    RE8 --> RE9["RE enters Badge/ID number<br/>in modal input
field<br/>See: Confirmation Modal Workflow"]
    RE9 --> Validate5{"Badge/ID<br/>provided?"}

    Validate5 -->|No| BadgeError["[ERROR] Badge/ID required<br/>Modal
shows error"]
    BadgeError --> RE9

    Validate5 -->|Yes| GenConfID["System generates:<br/>- confirmationId
via<br/>generateConfirmationId()"]
    GenConfID --> CreateConfData["ConfirmationData object:<br/>fullName:
RE.displayName<br/>badgeId: user input<br/>timestamp: formatTimestamp()
<br/>confirmationId: system generated<br/>confirmedBy:
RE.uid<br/>confirmedByEmail: RE.email<br/>confirmedByCompany: from
UserData<br/>confirmedAt: ISO timestamp"]
    CreateConfData --> StoreConf["storeConfirmation() executes:<br/>1.
getCaseData<br/>2. Find originalImageId mapping<br/>3. Add to
confirmations object<br/>4. updateCaseData() stores"]
    StoreConf --> AuditConf["auditService.logConfirmationCreation():<br/>-
action: confirm<br/>- workflowPhase: confirmation<br/>- confirmationId:
UUID<br/>- result: success"]
    AuditConf --> Phase5["PHASE 5: Confirmation Export<br/>(Reviewing
Examiner)"]

    Phase5 --> RE10["RE navigates to Case Export"]
    RE10 --> Check1["getCaseConfirmations()<br/>retrieves confirmations
object"]
    Check1 --> HasConf{"Confirmations<br/>exist?"}

    HasConf -->|No| NoExport["Export Confirmations button<br/>does not
appear"]
    NoExport --> End2([Process Ends])

    HasConf -->|Yes| ShowBtn["Export Confirmations button<br/>becomes
visible<br/>in export modal"]
    ShowBtn --> RE11["RE clicks<br/>Export Confirmations"]
    RE11 --> ConfExport["exportConfirmationData() executes:<br/>1.
getCaseDataWithManifest()<br/>2. Build unsigned JSON:<br/>metadata +
confirmations"]
    ConfExport --> CalcHash2["calculateSHA256Secure()<br/>computes SHA-256
over unsigned export data<br/>stores metadata.hash"]
    CalcHash2 --> SignConf["signConfirmationData() sends payload<br/>to
DATA_WORKER /api/forensic/sign-confirmation"]
    SignConf --> AttachSig["Attach signature metadata:
<br/>metadata.signatureVersion<br/>metadata.signature"]
    AttachSig --> DownloadConf["Browser downloads:<br/>confirmation-data-
[case]-<br/>[timestamp].json"]
    DownloadConf -->
AuditConfExport["auditService.logConfirmationExport():<br/>- action:
export<br/>- workflowPhase: confirmation<br/>- fileType: confirmation-
data<br/>- signature: present/valid/keyId"]
    AuditConfExport --> TransferConf["Transfer confirmation file to
OE<br/>via secure channels"]
    TransferConf --> Complete3([Phase 4-5 Complete])

    style BadgeError fill:#ff9900,color:#000
    style Complete3 fill:#90EE90,color:#000
```

Diagram 4: Import Confirmations & Final State

(Original Examiner)

Phase 6-7: Original examiner imports confirmations and marks images as confirmed.

```
flowchart TD
    Start([OE Confirmation Import Begins])

    Start --> Phase6["PHASE 6: Confirmation Import<br/>(Original
Examiner)"]

    Phase6 --> OE8["OE opens Striae"]
    OE8 --> OE9["OE navigates to Case Import"]
    OE9 --> OE10["OE selects confirmation JSON<br/>file received from RE"]
    OE10 --> Detect["isConfirmationDataFile():
<br/>filename.startsWith('confirmation-data')<br/>&&
filename.endsWith('.json')"]
    Detect --> ConfImportPreview["loadConfirmationPreview():<br/>- Parse
JSON<br/>- Display metadata summary:<br/>  - Case number<br/>  - RE
info<br/>  - Total confirmations<br/>  - Confirmation IDs"]
    ConfImportPreview --> OE11["OE reviews preview<br/>clicks Confirm
Import"]
    OE11 --> ConfImport["importConfirmationData():<br/>stages with
progress callback"]
    ConfImport --> Stage1["Stage 1: Read & parse
file<br/>JSON.parse(fileContent)"]
    Stage1 --> Stage2["Stage 2: Validate
hash<br/>validateConfirmationHash():<br/>- Remove metadata.hash<br/>-
Remove metadata.signature<br/>- Remove metadata.signatureVersion<br/>-
calculateSHA256Secure() compare"]
    Stage2 --> ValidHash{"Hash valid?"}

    ValidHash -->|No| ValidHashError["[BLOCKED] Hash mismatch<br/>File
corrupted or tampered"]
    ValidHashError --> ErrorEnd4([Import Failed])

    ValidHash -->|Yes| Stage3["Stage 3: Validate
signature<br/>validateConfirmationSignatureFile():<br/>- Signature
required<br/>- signatureVersion must be 2.0<br/>- Canonical payload
verification"]
    Stage3 --> ValidSig{"Signature validation<br/>passed?"}

    ValidSig -->|No| ValidSigError["[BLOCKED] Missing/invalid
signature<br/>Fail-closed import"]
    ValidSigError --> ErrorEnd5([Import Failed])

    ValidSig -->|Yes| Stage4["Stage 4: Validate
exporter<br/>validateExporterUid():<br/>- Fetch RE from USER_WORKER<br/>-
Check RE is not OE"]
    Stage4 --> ValidExporter{"Exporter validation<br/>passed?"}

    ValidExporter -->|No| ValidExporterError["[BLOCKED] Exporter does not
exist<br/>or same examiner as importer"]
    ValidExporterError --> ErrorEnd6([Import Failed])

    ValidExporter -->|Yes| Stage5["Stage 5: Validate
case<br/>checkExistingCase():<br/>Case must exist in user list"]
    Stage5 --> CaseExists{"Case exists<br/>in cases?"}

    CaseExists -->|No| CaseError["[BLOCKED] Case not found<br/>Can only
import confirmations<br/>for own cases"]
    CaseError --> ErrorEnd7([Import Failed])

    CaseExists -->|Yes| Stage6["Stage 6: Process confirmations<br/>For
each originalImageId:<br/>1. Map to currentImageId<br/>2. Fetch annotation
data"]
    Stage6 --> CheckDupe["Check if confirmationData<br/>already exists"]
    CheckDupe --> DupeExists{"Confirmation<br/>exists?"}

    DupeExists -->|Yes| Skip["[WARNING] Skip image<br/>Already has
confirmation"]
    Skip --> NextImage["Move to next image"]
    NextImage --> MoreImages{"More<br/>images?"}
    MoreImages -->|Yes| Stage6
```

```
    DupeExists -->|No| LinkCheck{"originalExportCreatedAt<br/>present?"}

    LinkCheck -->|No| LegacyMetaError["[ERROR] Missing forensic timestamp
link<br/>Legacy confirmation export rejected"]
    LegacyMetaError --> AddError["Add to result.errors"]

    LinkCheck -->|Yes| TimeCheck["Validate annotations unchanged:
<br/>annotationData.updatedAt<br/>must NOT be
after<br/>originalExportCreatedAt"]
    TimeCheck --> TimeValid{"Annotations<br/>valid?"}

    TimeValid -->|No| TimeError["[ERROR] Annotations modified<br/>after
original export"]
    TimeError --> AddError
    AddError --> NextImage

    TimeValid -->|Yes| StoreConfImport["storeConfirmationData():
<br/>Update annotation:
<br/>includeConfirmation=true<br/>confirmationData=imported confirmation"]
    StoreConfImport --> MarkReadOnly["Mark image as confirmed in UI:
<br/>isConfirmed derived from confirmationData<br/>edit functions
disabled"]
    MarkReadOnly --> UpdateImage["Push updated annotation<br/>to image in
case"]
    UpdateImage --> IncResult["result.confirmationsImported++
<br/>result.imagesUpdated++"]
    IncResult --> MoreImages

    MoreImages -->|No| Complete["All confirmations processed"]
    Complete --> AuditConfImport["auditService.logConfirmationImport():
<br/>- action: import<br/>- workflowPhase: confirmation<br/>- hashValid:
true<br/>- signature: present/valid/keyId<br/>- confirmationsImported:
count"]
    AuditConfImport --> Phase7["PHASE 7: Final State<br/>(Original
Examiner)"]

    Phase7 --> FinalCheck["Confirmed images:<br/>- Have
confirmationData<br/>- includeConfirmation remains true<br/>- Show
confirmed indicator in UI"]
    FinalCheck --> Immutable["Box annotations locked:<br/>- Cannot
edit<br/>- Cannot add new<br/>- Cannot toggle confirmation"]
    Immutable --> PDFReady["PDF generation includes:<br/>- Confirmation
details<br/>- RE credentials<br/>- Confirmation ID<br/>- Timestamp"]
    PDFReady --> Complete4([SUCCESS: Workflow Complete])

    style ValidHashError fill:#ff4444,color:#fff
    style ValidSigError fill:#ff4444,color:#fff
    style ValidExporterError fill:#ff4444,color:#fff
    style CaseError fill:#ff4444,color:#fff
    style LegacyMetaError fill:#ff4444,color:#fff
    style TimeError fill:#ff4444,color:#fff
    style Skip fill:#ffcc00,color:#000
    style Complete4 fill:#90EE90,color:#000
```

Confirmation Modal Workflow

Back to Diagram 3: Independent Review & Confirmation.

The confirmation modal controls badge entry and creation of the confirmation object that gets persisted to annotation data. Existing confirmations are rendered in read-only mode.

Primary behavior:

- Requires badge or ID input before submission.
- Builds a confirmation payload with examiner identity and timestamps.
- Clears transient errors as the user corrects input.
- Supports standard close paths (escape key, overlay click, close button).

For exact payload/canonical signing behavior after modal submission, see Manifest and

Confirmation Signing.

Key Validation Checkpoints

Security validations (blocking)

- Signed manifest presence (case ZIP): FORENSIC_MANIFEST.json must exist for case import.
- Manifest signature verification: signature must verify and manifestVersion must be 2.0.
- Confirmation signature verification: confirmation export must include valid metadata.signature and metadata.signatureVersion 2.0.
- Hash integrity: SHA-256 hash must match (confirmation hash excludes hash/signature fields during recomputation).
- Exporter verification: reviewing examiner must exist in user database.
- Self-confirmation prevention: RE UID must not equal OE UID.
- Case ownership: case must exist in OE case list for confirmation import.
- Annotation integrity: annotations cannot be modified after original export timestamp.
- Forensic linking requirement: originalExportCreatedAt is required for confirmation imports.
- Fail-closed policy: missing or invalid signature metadata blocks import.

Data validations (warning/skip)

- Duplicate detection: skip if confirmation already exists for an image.
- File format: only confirmation-data-*.json files are accepted.
- Metadata consistency: case number must match.

Canonical payload ordering and signature envelope details are defined in Manifest and Confirmation Signing.

Audit Logging Integration

All major actions are logged via auditService:

- logCaseExport: workflowPhase case-export
- logCaseImport: workflowPhase case-import
- logConfirmationCreation: workflowPhase confirmation, action confirm
- logConfirmationExport: workflowPhase confirmation, action export
- logConfirmationImport: workflowPhase confirmation, action import

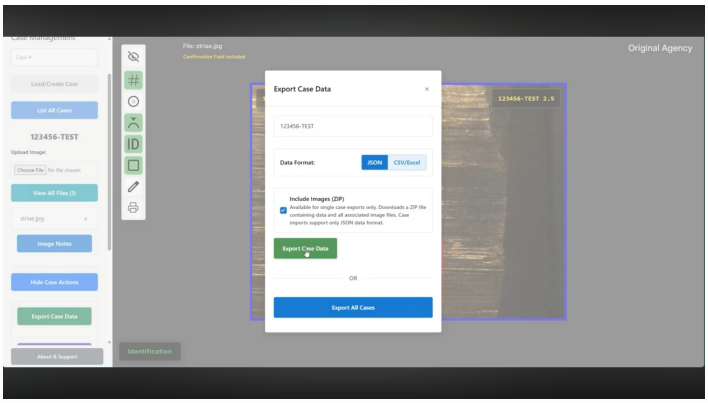
In v3.0.0+, import/export audit events also capture signature provenance (present, valid, keyId) for chain-of-custody evidence.

Screenshots

Legend

- OE = Original Examiner
- RE = Reviewing Examiner

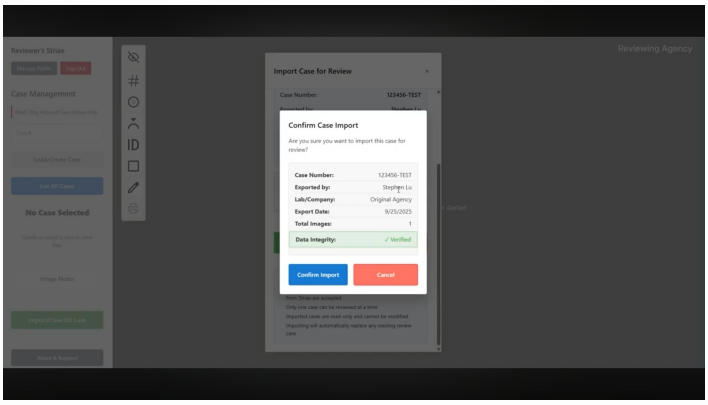
1. Original case export (OE)



Original case export (OE)

Figure 1. OE exports the case package with images and metadata, including forensic manifest hash generation and server-issued manifest signature.

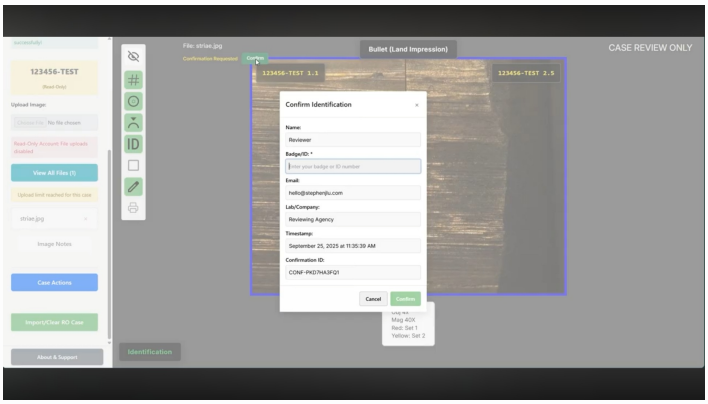
2. Original case import (RE)



Original case import (RE)

Figure 2. RE imports the package and confirms successful validation of package structure, signed forensic manifest, comprehensive integrity, and examiner identity.

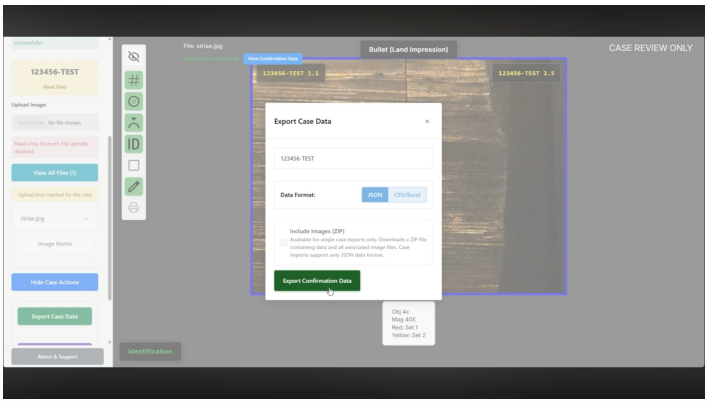
3. Confirmation modal (RE)



Confirmation modal (RE)

Figure 3. RE enters badge or ID in the confirmation modal, with required-field enforcement before confirmation is allowed.

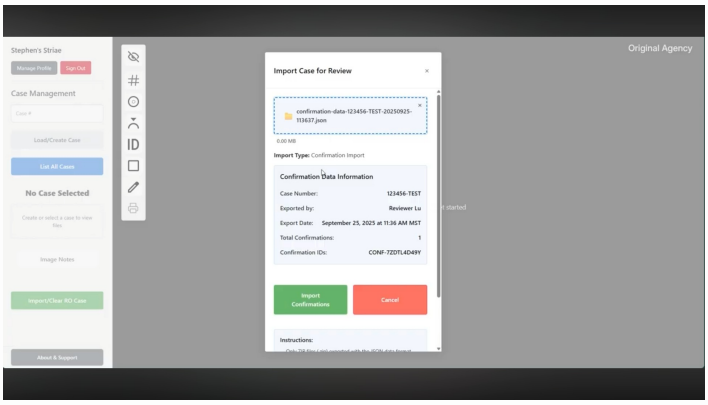
4. Confirmation export (RE)



Confirmation export (RE)

Figure 4. RE exports confirmation data as a dedicated JSON artifact containing confirmation records, integrity hash, and server-issued signature metadata.

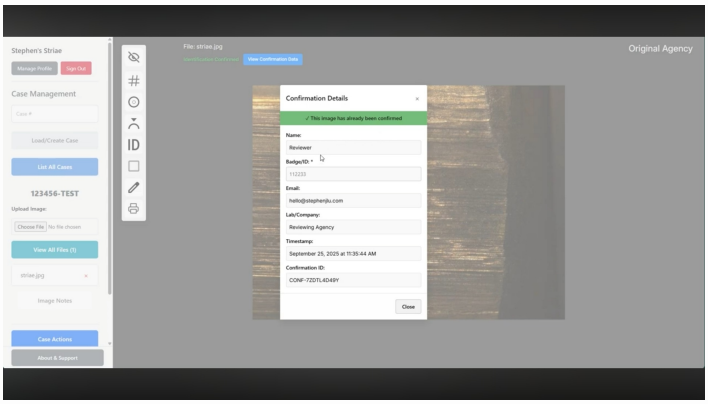
5. Confirmation import (OE)



Confirmation import (OE)

Figure 5. OE imports received confirmation data and the system validates hash, signature, source examiner, case ownership, duplicates, and forensic timestamp linkage.

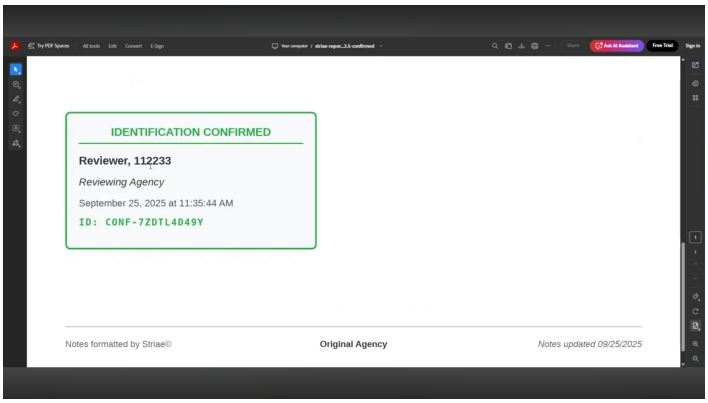
6. Confirmation review (OE)



Confirmation review (OE)

Figure 6. OE reviews applied confirmations, where confirmed images are marked and annotation editing is restricted.

7. Confirmation display on PDF (OE)



Confirmation display on PDF (OE)

Figure 7. Generated PDF displays confirmation details, including reviewer credentials, confirmation ID, and timestamp.

Related Documentation

- Manifest and Confirmation Signing
- API Reference
- Security Guide
- Audit Trail System
- Component Guide